Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота № 8
з курсу «Основи FE-технологій»
Тема: «Верстка сайту за допомогою HTML-5 і CSS-3 за зразком»

Виконала:
Студентка групи ІО-44
Савченко Руслана
Номер у списку курсу: 100
Перевірив:
Жереб Костянтин Анатолійович

Київ 2026

Мета роботи

Навчитися робити верстку сайту за зразком за допомогою HTML-5 і CSS-3 та створювати адаптивний дизайн.

Завдання

1. Зробити верстку сайту відповідно до свого варіанта (дизайн зберегти, текст і малюнки можуть відрізнятися від оригіналу).
2. *Зробити адаптивний дизайн розробленого сайту.

Номер 100 відповідає макету № 10: сайт аукціону з шапкою та навігацією, головним банером, каталогом лотів, блоком «Як це працює» (4 кроки), розділом «Про нас» і блоком відгуків. Окрім верстки, сайт отримав динамічну поведінку та простий серверний бекенд на Node.js: завантаження лотів із JSON, пошук, фільтри, сортування, таймери, ставки, реєстрацію та додавання лотів.

Виконання завдання

Використані інструменти та порівняльний аналіз

Семантичні теги HTML-5. Використано header, nav, main, section, article, blockquote, footer, а також списки ul та ol. Порівняно з верстkою на самих div, семантичні теги роблять структуру зрозумілою для розробника, пошукових систем і програм екранного читання.

CSS Grid і Flexbox. Grid застосовано для двовимірних сіток: картки аукціонів (4 колонки), кроки, блок «Про нас», відгуки. Flexbox застосовано для одного ряду або стовпця: шапка, меню, рядок «ціна та ставки». Застарілий спосіб із float не використовувався, бо він складніший у підтримці.

Змінні CSS (custom properties). Кольори задано в :root (--orange, --dark, --muted), тож зміна палітри потребує правки в одному місці.

Медіазапити та viewport. Тег meta viewport вмикає коректне масштабування на телефонах. Для адаптивності використано медіазапити max-width із брейкпоінтами 992, 768 і 480 px. Альтернатива — підхід mobile-first (min-width), який зручніший для нових проєктів, але для верстки за готовим десктопним зразком природніше йти від великого екрана до малого.

JavaScript. Каталог лотів створюється динамічно: дані завантажуються методом fetch() у форматі JSON і обробляються ланцюжком .then() та .catch() (як у лабораторній №6), а картки будуються через document.createElement() (як у лабораторній №4). Події click, input, change і submit обробляються через addEventListener, для карток застосовано делегування подій (лабораторна №5). Форми реєстрації, входу, ставки та додавання лота перевіряються регулярними виразами. Таймери зворотного відліку працюють на setInterval, обрані лоти та користувач зберігаються в localStorage, а діалогові вікна створено елементом <dialog>.

Серверна частина (Node.js). Файл server.js — простий сервер на вбудованому модулі http без зовнішніх залежностей. Він віддає сторінку та надає REST API: GET /api/auctions, POST /api/register, POST /api/login, POST /api/auctions/:id/bid, POST /api/auctions. Дані зберігаються у файлі data/db.json, паролі — у вигляді

хешу scrypt із сіллю, а ставки й додавання лотів доступні лише з токеном сесії. Ті самі регулярні вирази перевіряються і в браузері, і на сервері. Альтернативи: Express спрощує маршрутизацію, але потребує встановлення пакетів; база даних (SQLite чи MongoDB) надійніша за JSON-файл, проте для навчальної роботи надмірна.

Режим без сервера. Якщо API недоступне (наприклад, на GitHub Pages), сторінка бере лоти з data/auctions.json і працює в демо-режимі: ставки та нові лоти зберігаються лише в браузері.

Шрифт і зображення. Шрифт Mulish підключено через Google Fonts. Зображення товарів замінено емодзі, що дозволено умовою завдання й не потребує окремих файлів.

Альтернативи. Bootstrap або Tailwind дають готові класи й швидше розробляються, але для навчальної роботи обрано чистий CSS, щоб продемонструвати власну верстку та адаптивність.

Хід роботи

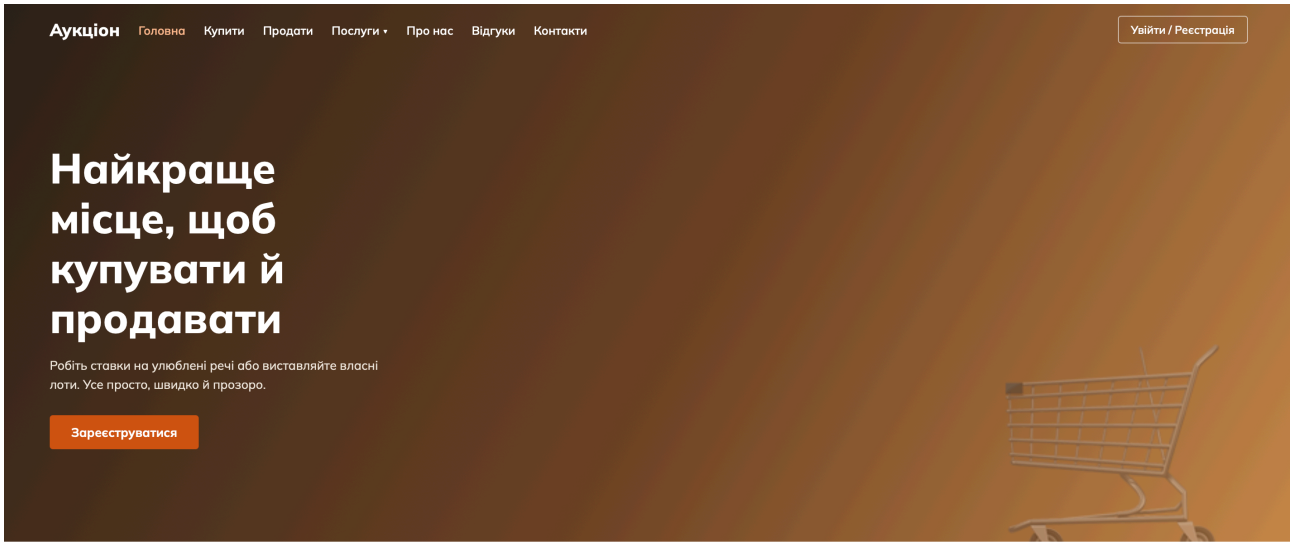
1. Проаналізовано макет № 10 і розбито його на блоки: банер із меню, аукціони, «Як це працює», «Про нас», відгуки, підвал.
2. Створено index.html із семантичною розміткою; у <title> вказано ПІБ, групу, номер лабораторної та варіант.
3. У style.css описано змінні, сітки Grid і Flexbox, підменю «Послуги», діалогові вікна та форми. Для адаптивності додано три медіазапити: на 992 px картки та кроки стають у 2 колонки, на 768 px з'являється меню-бургер, на 480 px усе йде в одну колонку.
4. У script.js реалізовано завантаження лотів через fetch, пошук, фільтр за категоріями, сортування, обрані лоти, таймери, а також діалоги ставки, реєстрації та додавання лота з перевіркою регулярними виразами.
5. У server.js написано сервер і REST API, дані зберігаються у data/db.json.
6. Запуск із сервером: node server.js, після чого сторінка доступна на http://localhost:3000. Без сервера сторінка відкривається в демо-режимі.
7. Перевірено роботу сторінки в адаптивному режимі DevTools на ширині 1200, 900, 700 і 375 px.
8. Код завантажено в репозиторій GitHub, сайт опубліковано на GitHub Pages.

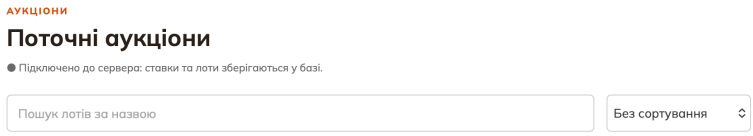
Посилання на репозиторій та опублікований проєкт

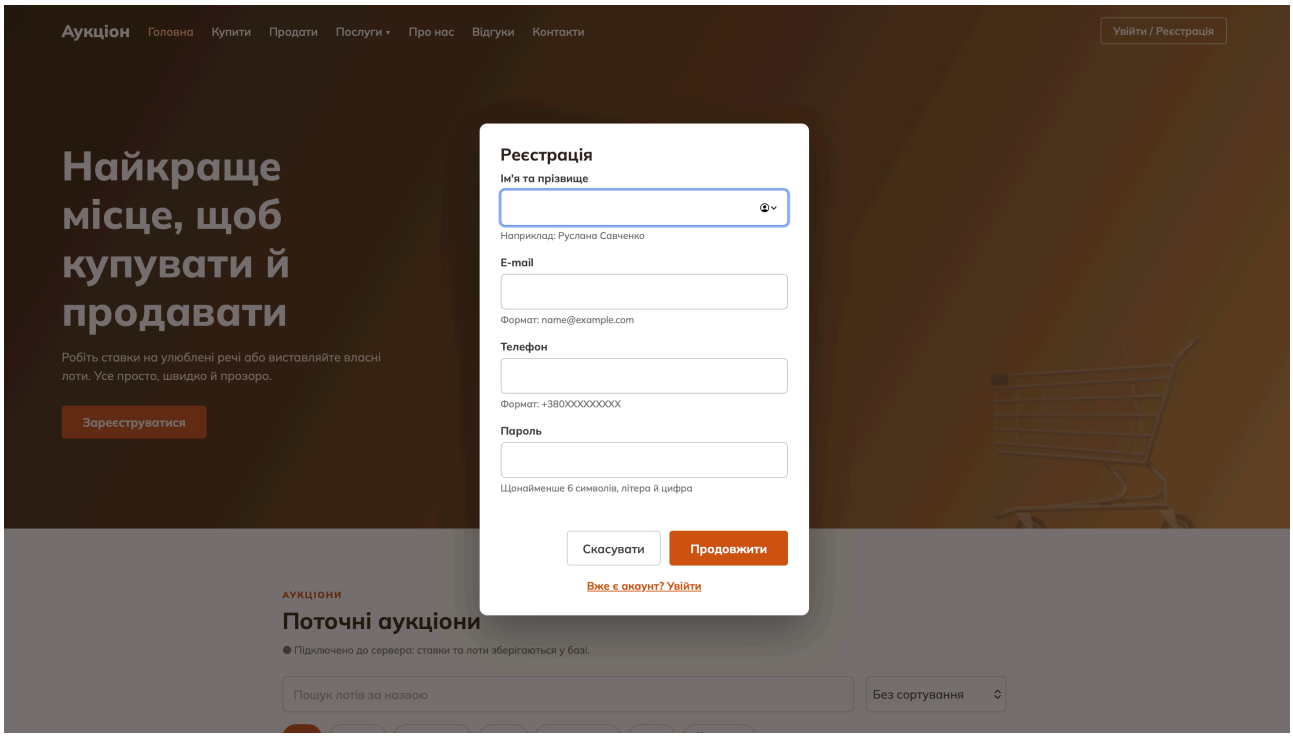
Проєкт у GitHub: https://github.com/sunamico/Front-end/tree/main/lab8

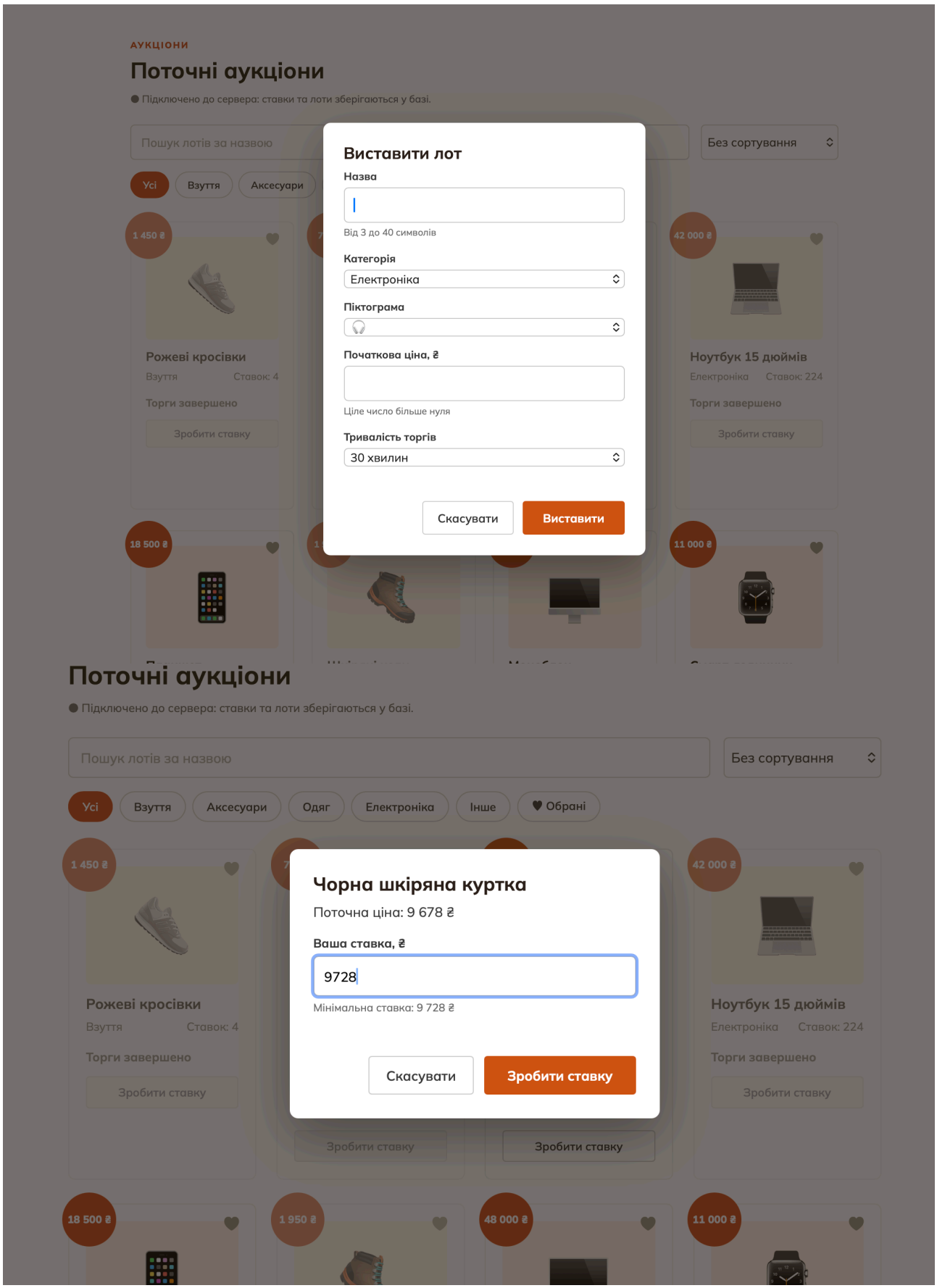
Створений сайт (GitHub Pages): https://sunamico.github.io/Front-end/lab8/

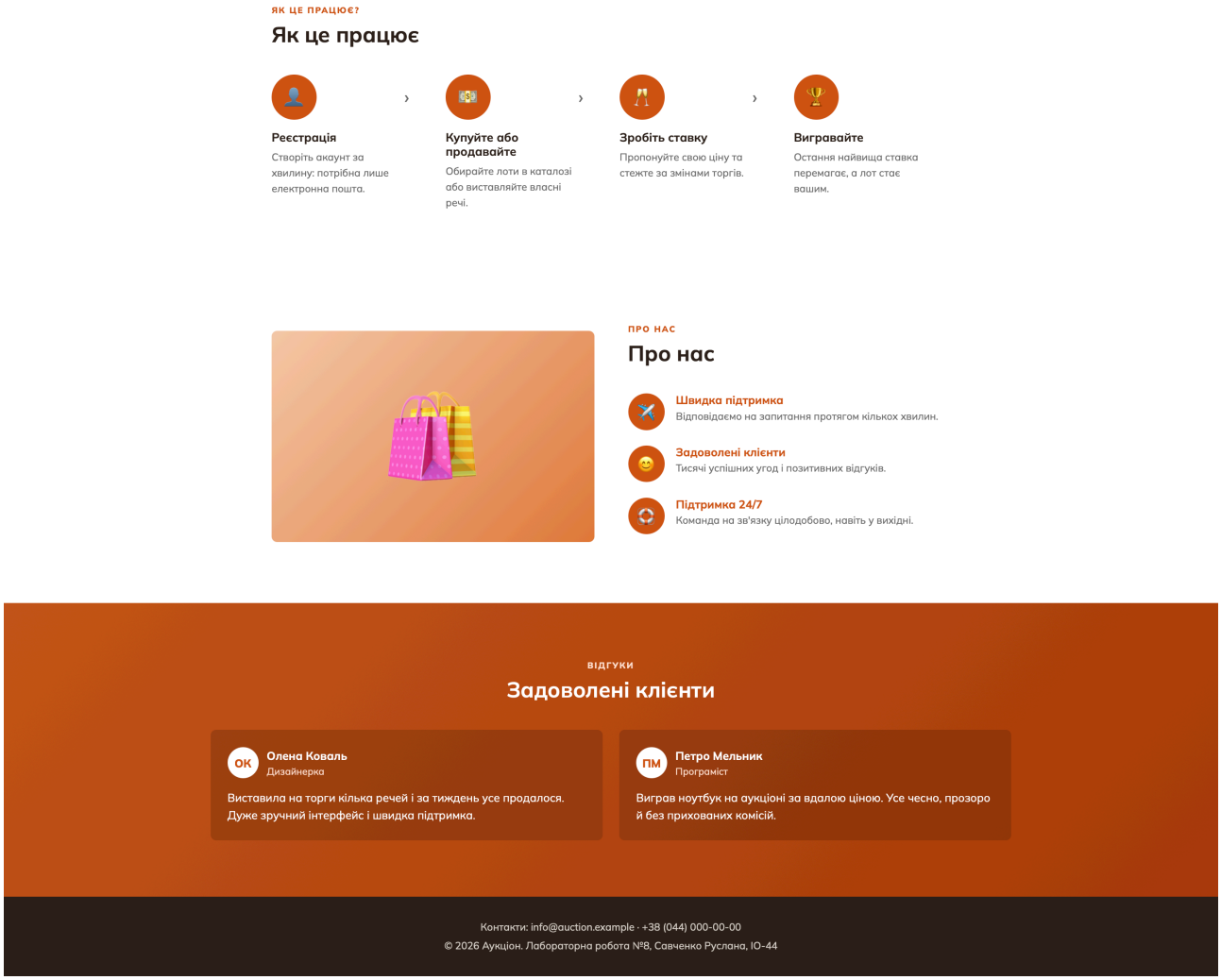
Результат виконання








АУКЦІОНИ
Поточні аукціони
Підключено до сервера: ставки та лоти зберігаються у базі.
Пошук лотів за назвою
Без сортування
Усі
Взуття
Аксесуари
Виставити лот
Назва
Від 3 до 40 символів
Категорія
Електроніка
Піктограма
Початкова ціна, ₴
Ціле число більше нуля
Тривалість торгів
30 хвилин
Скасувати
Виставити
1 450 ₴
Рожеві кросівки
Взуття
Ставок: 4
Торги завершено
Зробити ставку
42 000 ₴
Ноутбук 15 дюймів
Електроніка
Ставок: 224
18 500 ₴
11 000 ₴
Поточні аукціони
Підключено до сервера: ставки та лоти зберігаються у базі.
Пошук лотів за назвою
Без сортування
Усі
Взуття
Аксесуари
Одяг
Електроніка
Інше
Обрані
Чорна шкіряна куртка
Поточна ціна: 9 678 ₴
Ваша ставка, ₴
9728
Мінімальна ставка: 9 728 ₴
Скасувати
Зробити ставку
1 950 ₴
48 000 ₴



Висновки

У ході лабораторної роботи № 8 я зробила верстку сайту за зразком за допомогою семантичних тегів HTML-5 і стилів CSS-3 (Grid, Flexbox, змінні CSS) та зробила її адаптивною за допомогою медіазапитів. Сайт отримав динамічну поведінку на JavaScript: завантаження даних із JSON через fetch і проміси, побудову карток через DOM, обробку подій, перевірку форм регулярними виразами, таймери та збереження стану в localStorage. Також я створила простий сервер на Node.js із REST API, реєстрацією, ставками та додаванням лотів. Код збережено в репозиторії GitHub, а сайт опубліковано на GitHub Pages, де він працює в демо-режимі без сервера.